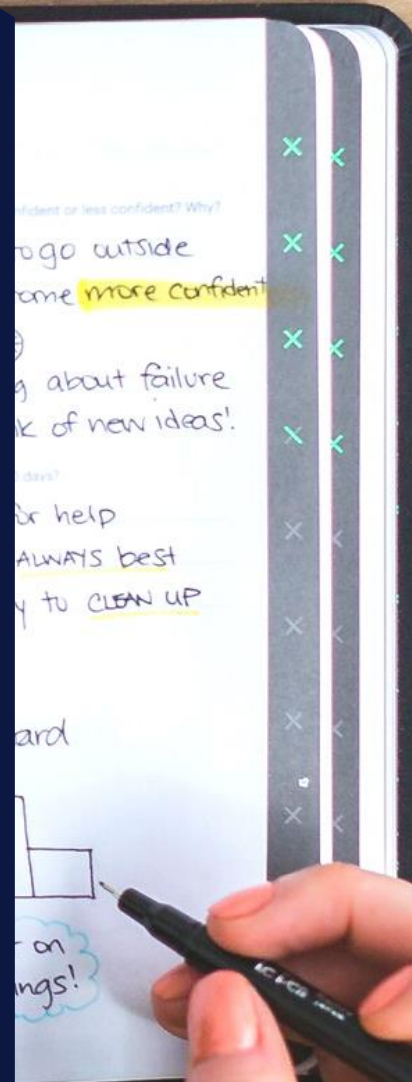




Tarea # 1

Unidad 4. Despliegue y
publicación de
aplicaciones



Actividades

Actividad: Despliegue del proyecto de la Unidad 3 (MercApp) en MongoDB Atlas, Railway, Netlify y publicación de micrositio en GitHub Pages

Objetivo General

Desplegar en Internet el proyecto desarrollado en la Unidad 3 (SPA con Vue 3 + API propia), realizando el aprovisionamiento de la base de datos en MongoDB Atlas, el despliegue del back-end en Railway, el despliegue del front-end en Netlify y la publicación de un micrositio del proyecto en GitHub Pages, garantizando HTTPS, CORS correctamente configurado, uso de variables de entorno y artefactos de build optimizados.

Requisitos Previos

- Proyecto de la Unidad 3 operativo (SPA + API), con repositorio público en GitHub y README.md.
- Cuentas activas en MongoDB Atlas, Railway, Netlify y GitHub Pages.
- Archivo .env.example (sin credenciales) y gitignore de entornos.
- Estructura mínima estandarizada:
 - Back-end: package.json con scripts dev y start, servidor leyendo process.env.PORT.
 - Front-end: variables VITE_* para endpoints (p. ej., VITE_API_URL).

Descripción general

La actividad se centra exclusivamente en el despliegue del proyecto de la Unidad 3. El estudiantado ejecutará cuatro fases y entregará evidencias y URLs funcionales.

Fase 1 — Base de datos (MongoDB Atlas)

- Crear Organización/Proyecto y un cluster gratuito (M0) en región cercana al back-end.
- Crear usuario de BD y habilitar acceso de red según la política del curso.
- Obtener la cadena de conexión mongodb+srv://... (Drivers → Node.js) y configurarla como MONGODB_URI en el back-end.

- Verificar conectividad con MongoDB Compass (TLS, credenciales, pruebas de lectura/escritura).

Fase 2 — Back-end (Railway)

- Conectar el repositorio de la API y permitir la detección automática del stack (Nixpacks).
- Configurar variables: MONGODB_URI, PORT, y orígenes de CORS (p. ej., FRONTEND_URL/NETLIFY_URL).
- Desplegar y validar un endpoint de salud y operaciones CRUD mínimas.

Fase 3 — Front-end (Netlify)

- Definir VITE_API_URL apuntando a la URL pública de la API en Railway.
- Ejecutar npm run build y publicar dist/ (manual o CI).
- Verificar funcionamiento de rutas (SPA), consumo del API y ausencia de errores de CORS.

Fase 4 — Micrositio del proyecto (GitHub Pages)

- Crear un sitio estático en /docs del repositorio (o en repo separado) con:
- Resumen técnico del sistema y arquitectura de despliegue.
- Guía breve de ejecución local y enlaces a Netlify (frontend) y a la URL base del API (Railway).
- Tabla corta de endpoints (métodos, rutas, ejemplo de request/response).
- Activar Pages y validar HTTPS y accesibilidad pública

Tareas Obligatorias

Nº	Descripción de la tarea obligatoria
1	Normalizar el repositorio del proyecto de la Unidad 3: estructura coherente, README con instrucciones de build/despliegue, .env.example, .gitignore y limpieza de dependencias.
2	Crear Organización/Proyecto y clúster M0 en MongoDB Atlas en una región cercana al futuro back-end.
3	Crear usuario de base de datos con rol adecuado; configurar IP allowlist según lineamientos del curso y confirmar TLS activo.

4	Obtener la cadena de conexión (Drivers → Node.js) y validarla con MongoDB Compass ejecutando una inserción y una consulta de prueba.
5	Parametrizar el back-end: leer MONGODB_URI y PORT desde variables de entorno, exponer /health, y definir scripts dev/start en package.json.
6	Desplegar la API en Railway desde GitHub: habilitar build con Nixpacks, configurar MONGODB_URI, PORT, y orígenes CORS (FRONTEND_URL/NETLIFY_URL), ejecutar primer deploy.
7	Verificar la API en producción: probar CRUD de productos/categorías con datos de ensayo, revisar logs en Railway y registrar incidencias.
8	Preparar el front-end (Vue 3): definir VITE_API_URL apuntando a la API en producción, ejecutar npm run build y validar con npm run preview.
9	Desplegar el front-end en Netlify: publicar la carpeta dist/ (manual o CI), configurar variables de entorno de build si aplica y comprobar enrutamiento de SPA.
10	Validar y ajustar CORS: confirmar que el preflight OPTIONS se resuelve; restringir orígenes al dominio público de Netlify y redeploy de la API si hubo cambios.
11	Publicar micrositio en GitHub Pages (carpeta /docs o repo dedicado) con arquitectura de despliegue, documentación breve de endpoints y enlaces a Netlify/Railway.
12	Entregar evidencias finales: Informe con URLs públicas (API, SPA y Pages), capturas de pruebas CRUD y export del repo en .zip.

INSTRUCCIONES PARA LA REALIZACIÓN DE LA ACTIVIDAD DE LA UNIDAD 4

1. Crear un repositorio en GitHub y trabajar de forma síncrona con el proyecto
2. Generar un archivo README.md indicando los datos del estudiante, las funcionalidades implementadas e instrucciones básicas de uso
3. Coloca la URL de su repositorio de GitHub en un archivo readme.txt dentro de la carpeta principal del proyecto
4. Una vez concluido su desarrollo, **comprimir la carpeta en un archivo ZIP y enviar al AVAC** con el siguiente formato “Actividad4-Apellido1-Nombre1.zip”.

Rúbrica:

Criterios	Bajo (0%)	Regular (30%)	Medio (60%)	Alto (100%)	Sub-puntaje máximo
1) Despliegue y operatividad end-to-end (Railway + Netlify + Atlas + Pages)	API/SPA no accesibles; errores de CORS que bloquean; BD no conectada; rutas SPA rotas; URLs públicas ausentes. 0	Al menos 1 componente publicado pero con fallos que impiden el flujo (p. ej., SPA carga pero API falla o CORS interrumpe); sin health check; GitHub Pages ausente. 1.8	API en Railway y SPA en Netlify accesibles con CRUD parcial; CORS resuelto parcialmente; rutas SPA con incidencias menores; GitHub Pages publicado pero desactualizado o sin enlaces completos. 3.6	CRUD completo, búsqueda/filtros; routing SPA correcto (fallback configurado), API con health, CORS restringido al dominio de Netlify, Atlas operativo con pruebas; GitHub Pages con enlaces a Netlify/Railway y descripción breve; todo bajo HTTPS. 6	6
2) Seguridad y configuración de entorno (secretos, CORS, HTTPS, acceso a BD)	Datos sensibles en el repositorio; sin .env.example; HTTP; allowlist de Atlas inapropiada; sin cabeceras de seguridad. 0	Variables de entorno parcialmente; .env.example ausente; CORS permisivo (“*”) con credenciales; allowlist amplia sin justificación; sin verificación TLS/Compass. 0.9	Variables y secretos bien gestionados (no versionados) y .env.example; CORS restringido parcialmente; HTTPS activo; evidencia básica de verificación TLS/Compass; allowlist temporal documentada. 1.8	Gestión robusta: secretos fuera del repo, .env.example, uso de npm ci; CORS estrictamente por dominio, HTTPS y cabeceras (p. ej., Helmet); rate-limiting/sanitización básicos; allowlist mínima y evidencias de pruebas TLS/Compass. 3	3.0
3) Observabilidad, rendimiento y documentación de despliegue (logs, build, Pages, README)	Sin revisión de logs; build no optimizado (modo dev); sin Pages; README insuficiente. 0	Revisión ocasional de logs; build de producción dudoso; Pages con información mínima; README parcial y sin enlaces. 0.9	Logs consultados y capturas de pruebas; build de producción (artefactos dist) y SPA funcional; GitHub Pages con arquitectura y endpoints; README con instalación/ejecución y enlaces básicos.	Evidencias sistematizadas (capturas/links); uso de logs para diagnóstico; mejoras de rendimiento verificadas (p. ej., revisión con Lighthouse); SPA con redirecciones/headers cuando aplica; Pages	3.0

			1.8	con diagrama, endpoints, versiones; README completo con enlaces a Netlify/Railway/Pages y checklist final. 3	
Totales					12